

AI SDK

Integrate any model provider with a single API

A TypeScript-first toolkit for building AI-powered applications, conversational interfaces, and autonomous agent loops across modern web frameworks and runtimes.

docs [ai-sdk.dev](#)

license [Apache 2.0](#)

monorepo [Turborepo](#)

package manager [pnpm v10](#)

orm [Drizzle ORM](#)

Overview

This repository is a monorepo powered by [Turborepo](#) and [pnpm](#), hosting the core **AI SDK** (`ai`), first-party model providers, framework UI adapters, developer tooling, and database persistence layers.

Whether you are crafting real-time streaming chatbots, integrating structured schema extraction, orchestrating multi-step autonomous tool-calling agents, or connecting persistent state with PostgreSQL via Drizzle ORM, this toolkit provides unified, type-safe abstractions designed to run seamlessly on Node.js, edge runtimes, and the browser.

Key Features

- **Unified Provider API:** Seamlessly swap between OpenAI, Anthropic, Google Generative AI, DeepSeek, Amazon Bedrock, Azure, Cohere, Cerebras, and more without rewriting application logic.
- **Robust Text & Stream Generation:** First-class support for `generateText` and `streamText` with backpressure control and streaming chunk abstractions.
- **Type-Safe Structured Outputs:** Enforce and validate model responses against Zod schemas using `Output.object` and `generateObject`.

- **Autonomous Agent Workflows:** Compose dynamic tool loops using `ToolLoopAgent` with native tool execution, shell sandboxes, and multi-turn reasoning.
 - **Framework UI Adapters:** High-performance hooks (`useChat`, `useCompletion`, `useAssistant`) tailored for React, Next.js, Svelte, Vue, and Angular.
 - **Database & Persistence Layer:** Integrated Drizzle ORM with PostgreSQL / Neon Serverless Postgres for schema migrations, conversation history, and user state.
-

Repository Structure

```

.
├── packages/
│   ├── ai/                # Core AI SDK engine and universal interfaces
│   ├── amazon-bedrock/    # AWS Bedrock provider adapter
│   ├── angular/           # Angular UI bindings and service hooks
│   ├── anthropic/         # Anthropic (Claude) provider adapter
│   ├── assemblyai/        # AssemblyAI audio and transcription provider
│   ├── azure/             # Azure OpenAI provider adapter
│   ├── baseten/           # Baseten model deployment provider
│   ├── black-forest-labs/  # Black Forest Labs image generation provider
│   ├── cerebras/          # Cerebras high-speed inference provider
│   ├── codemod/           # AST transformation scripts and migration utilities
│   ├── cohere/            # Cohere provider adapter
│   ├── deepgram/          # Deepgram speech-to-text and audio adapter
│   ├── deepinfra/         # DeepInfra serverless inference provider
│   ├── deepseek/          # DeepSeek reasoning and chat model provider
│   ├── devtools/          # Developer inspection tools and telemetry
│   ├── elevenlabs/        # ElevenLabs voice generation provider
│   └── fal/               # Fal.ai generative media provider
├── examples/              # Production-ready starter apps & blueprints
│   ├── ai-core/           # Minimal Node.js / TypeScript usage patterns
│   ├── next/              # Next.js App Router integrations
│   ├── next-agent/        # Autonomous tool agent implementations
│   ├── angular/           # Angular chat application
│   ├── hono/              # Ultra-fast edge API with Hono
│   ├── mcp/               # Model Context Protocol integration examples
│   ├── sveltekit-openai/   # SvelteKit with OpenAI streaming
│   └── ...                # Express, Fastify, Nest, Nuxt, and more
├── src/                   # Database schema and Drizzle ORM persistence
└── db.ts                 # PostgreSQL connection pool and Drizzle client

```

```
|   ├── schema.ts           # Database table definitions and inference type
|   └── index.ts            # Example CRUD workflow demonstrating Drizzle ORM
└── drizzle/               # Generated SQL schema migrations
    ├── content/           # Documentation content and reference guides
    └── drizzle.config.ts   # Drizzle Kit configuration
```

Quick Start

Prerequisites

- **Node.js:** `v18.0.0`, `v20.0.0`, or `v22.0.0+`
- **pnpm:** `v10.11.0` or higher (`corepack enable` or `npm install -g pnpm@10`)
- **PostgreSQL Database** (optional, for persistent storage workflows): e.g. Neon Serverless Postgres

Installation

Clone the repository and install dependencies from the root directory:

```
git clone https://github.com/RISE-DEFENSE-SYSTEMS-AI/ai.git
cd ai
pnpm install
```

Build all packages across the monorepo:

```
pnpm build
```

Core Usage

1. Generating Text

```
import { generateText } from 'ai';
import { openai } from '@ai-sdk/openai';

const { text } = await generateText({
  model: openai('gpt-5'),
```

```
    prompt: 'Explain quantum computing in three concise sentences.',
  });

  console.log(text);
```

2. Structured Outputs with Type Safety

```
import { generateText, Output } from 'ai';
import { openai } from '@ai-sdk/openai';
import { z } from 'zod';

const { output } = await generateText({
  model: openai('gpt-5'),
  output: Output.object({
    schema: z.object({
      title: z.string(),
      summary: z.string(),
      actionItems: z.array(z.string()),
    }),
  }),
  prompt: 'Summarize the release notes for version 2.0.',
});

console.log(output.actionItems);
```

3. Tool-Calling Agents

```
import { ToolLoopAgent } from 'ai';
import { openai } from '@ai-sdk/openai';
import { z } from 'zod';

const weatherAgent = new ToolLoopAgent({
  model: openai('gpt-5'),
  system: 'You are an intelligent assistant with access to live weather data.',
  tools: {
    getWeather: {
      description: 'Get current weather conditions for a location',
      parameters: z.object({ city: z.string() }),
      execute: async ({ city }) => ({
```

```
    city,  
    temperature: '72°F',  
    condition: 'Sunny',  
  }},  
},  
},  
});
```

4. Database & Persistence Layer (PostgreSQL + Drizzle ORM)

The repository includes a ready-to-use Drizzle ORM setup for storing user data, conversation sessions, and agent memories.

1. Configure your database connection in `.env`:

```
DATABASE_URL=postgresql://user:password@ep-sample-pooler.region.neon.tech
```

2. Generate and apply migrations:

```
pnpm db:generate  
pnpm db:migrate
```

3. Run the example database operations:

```
npx tsx src/index.ts
```

Monorepo Development Scripts

Command	Description
<code>pnpm build</code>	Build all packages and examples via Turborepo
<code>pnpm build:packages</code>	Build only the core packages (<code>@ai-sdk/*</code> , <code>ai</code>)
<code>pnpm build:examples</code>	Build only the example projects (<code>examples/*</code>)
<code>pnpm dev</code>	Start development watch mode across the workspace
<code>pnpm test</code>	Run test suites across all packages using Vitest

Command	Description
<code>pnpm lint</code>	Lint code using ESLint across packages
<code>pnpm type-check</code>	Validate TypeScript types across the monorepo
<code>pnpm prettier-fix</code>	Format source code, markdown, and config files
<code>pnpm db:generate</code>	Generate SQL migration files from Drizzle schema
<code>pnpm db:migrate</code>	Apply pending SQL migrations to the database
<code>pnpm changeset</code>	Create a new changeset for versioning and package release

Contributing

Contributions are welcome! Please read through our [Contribution Guidelines](file:///Users/rdsairevops/ai/CONTRIBUTING.md) to understand the local development setup, coding standards, changeset workflow, and PR submission conventions.

- Follow the [Code of Conduct](file:///Users/rdsairevops/ai/CODE_OF_CONDUCT.md).
 - When modifying package dependencies, execute `pnpm update-references` to maintain TypeScript project reference consistency.
 - Ensure all tests and linting passes before submitting pull requests.
-

License

This project is licensed under the **Apache License 2.0**. See the [LICENSE](file:///Users/rdsairevops/ai/LICENSE) file for details.